

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:
«Операционные системы»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

«Планировщики ввода-вывода в Linux»

Выполнили:

Жук Анастасия Валерьевна
студентка группы N3248



Проверил:

Кича Игорь Владимирович
инженер факультета БИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург
2024 г.

СОДЕРЖАНИЕ

Содержание	3
Введение	4
Основная часть.....	5
Планировщики ввода-вывода	5
Практика.....	7
Заключение.....	10
Список использованных источников.....	11

ВВЕДЕНИЕ

Цель работы – провести тестирование и найти лучший планировщик ввода-вывода среди других.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить существующие планировщики;
2. Написать тесты, позволяющие протестировать работу планировщика;
3. Провести сравнительный анализ результатов.

ОСНОВНАЯ ЧАСТЬ

Планировщики ввода-вывода

В ранних ядрах Linux существовал только один планировщик — Linus Elevator. Он был слишком примитивным, из-за чего начиная с ядра версии 2.6 появились еще три планировщика, часть из которых уже не существует. Таким образом, **устаревшими** планировщиками ввода-вывода считаются NOOP, CFQ, Deadline, Anticipatory, **альтернативными** - none, mq-deadline, kyber, BFQ (Budget Fair Queue).

NOOP - самый простой планировщик, обладающий минимальными возможностями и выполняющий только простые операции объединения и сортировки, но зато потребляющий минимум ресурсов. NOOP представляет собой очередь FIFO и предназначен в основном для работы с не дисковыми устройствами (ОЗУ или flash-диск)/специализированными решениями, которые уже имеют свой собственный планировщик I/O. В этом случае его простота имеет преимущество перед остальными алгоритмами.

CFQ - планировщик I/O, основная цель которого — обеспечение справедливого распределения пропускной способности дискового ввода-вывода для всех процессов, которые запрашивают операцию чтения и записи. Каждому процессу назначается своя очередь запросов ввода/вывода. Каждой очереди затем присваивается квант времени. Планировщик циклически обходит все процессы и обслуживает каждый из них, пока не закончится очередь либо не истечет заданный квант времени. Если очередь закончилась раньше, чем истек выделенный для нее квант времени, планировщик подождет (по умолчанию 10 мс) и, в случае напрасного ожидания, перейдет к следующей очереди. В рамках каждой очереди чтение имеет приоритет над записью.

Deadline - в данном планировщике каждый запрос связан со временем истечения. По умолчанию время истечения составляет 500 миллисекунд в будущем для запросов на чтение и 5 секунд в будущем для запросов на запись. Он работает аналогично Linus Elevator в том, что он поддерживает очередь запросов, отсортированную по физическому расположению на диске. Когда новый запрос отправляется в отсортированную очередь, планировщик ввода-вывода Deadline выполняет слияние и вставку, как Linus Elevator. Однако планировщик Deadline также вставляет запрос во вторую очередь, которая зависит от типа запроса. Запросы на чтение сортируются в специальную очередь FIFO на чтение, а запросы на запись вставляются в специальную очередь FIFO на запись. Хотя обычная очередь сортируется по секторам на диске, эти очереди хранятся в порядке FIFO (фактически они сортируются по времени). Следовательно, новые запросы всегда добавляются в хвост очереди. При нормальной работе планировщик ввода-вывода Deadline извлекает запросы из головы отсортированной очереди в очередь отправки. Затем очередь отправки подается на диск.

Anticipatory - основан на Deadline, его алгоритм позволяет минимизировать перемещение головки по диску. Для этого перед запросом вводится некая управляемая задержка, при помощи которой достигается переупорядочивание и объединение операций обращения к диску. Поэтому есть вероятность того, что предыдущий запрос успеет получить нужные

данные до того, как головка диска будет вынуждена перейти на новый сектор. Результатом работы Anticipatory может быть увеличение времени задержки выполнения операций ввода/вывода, поэтому его лучше всего использовать на клиентских машинах с медленной дисковой подсистемой, для которых более важна интерактивность работы, чем задержки ввода/вывода. Этот алгоритм использовался по умолчанию в ядрах 2.6.0 - 2.6.17.

none - планировщик ввода-вывода без операций с несколькими очередями. Не переупорядочивает запросы и имеет минимальные расходы. Идеально подходит для быстрых устройств произвольного ввода-вывода, таких как SSD или NVME.

mq-deadline - реализация deadline с использованием blk-mq - способа передачи запроса драйверу - со следующим принципом работы: запрос сначала попадает в программную очередь (количество этих очередей = количество ядер процессора). После прохождения программной очереди запрос попадает в очередь отправки (количество очередей отправки зависит от драйвера устройства, который может поддерживать от 1 до 2048 очередей). Так как работа планировщика осуществляется на уровне программных очередей, то запрос из любой программной очереди может попасть в любую очередь отправки, предоставляемую драйвером.

kyber - был написан для работы с быстрыми устройствами. Используя две очереди — запросы на запись и на чтение, — kyber отдает приоритет запросам на чтение, перед запросами на запись. Алгоритм измеряет время завершения каждого запроса и корректирует фактический размер очереди для достижения установленных в настройках задержек.

BFQ - основан на алгоритме от CFQ, но содержит улучшения. В конфигурации по умолчанию он больше нацелен на обеспечение минимальной задержки, а не на достижение максимальной пропускной способности. Этот планировщик обладает относительно высокими расходами на операцию, поэтому он не идеален для устройств с медленными процессорами или устройствами ввода-вывода с высокой пропускной способностью.

Практика

```
root@Ubuntu:~# cat /sys/block/sda/queue/scheduler
none [mq-deadline]
root@Ubuntu:~# sudo modprobe bfq
root@Ubuntu:~# cat /sys/block/sda/queue/scheduler
none [mq-deadline] bfq
root@Ubuntu:~# sudo modprobe kyber
modprobe: FATAL: Module kyber not found in directory /lib/modules/6.8.0-48-generic
root@Ubuntu:~# find /lib/modules/$(uname -r)/ -name '*kyber*'
/lib/modules/6.8.0-48-generic/kernel/block/kyber-iosched.ko.zst
root@Ubuntu:~# sudo modprobe kyber-iosched
root@Ubuntu:~# cat /sys/block/sda/queue/scheduler
none [mq-deadline] bfq kyber
```

Рисунок 1 – Установка планировщиков и проверка их наличия

`modprobe` – команда Linux для загрузки модулей ядра.

`cat /sys/block/sda/queue/scheduler` – вывод доступных планировщиков ввода-вывода в системе.

```
root@Ubuntu:~# sudo apt install hdparm
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
hdparm is already the newest version (9.65+ds-1build1).
hdparm set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 66 not upgraded.
```

Рисунок 2 – Установка hdparm

`hdparm` — это программа командной строки для Linux, которая используется для работы с дисковыми устройствами и жёсткими дисками.

С помощью этой команды можно:

- получать статистику о жёстком диске;
- изменять интервалы записи;
- управлять акустикой;
- настраивать DMA.

Также `hdparm` можно использовать как простой инструмент для тестирования производительности. Именно эта функция этой команды будет использоваться в дальнейшем.

Листинг 1 – test.sh

```
DISC="sda"; \  
cat /sys/block/$DISC/queue/scheduler; \  
for T in none mq-deadline bfq kyber; do \  
    echo $T > /sys/block/$DISC/queue/scheduler; \  
    cat /sys/block/$DISC/queue/scheduler; \  
    sync && /sbin/hdparm -tT /dev/$DISC && echo "----"; \  
    sleep 15; \  
done
```

test.sh – скрипт. Он прогоняется по перечисленным в нём планировщикам и первой строчкой в цикле выбирает текущий в качестве активного. Далее командой sync завершаются все операции записи данных на диск и применяется утилита hdparm в двух разных режимах: -t тестирует скорость последовательного чтения непосредственно с указанного диска (измеряет реальную пропускную способность), -Т тестирует скорость чтения данных из кэша (RAM, не с диска) (измеряет, насколько быстро процессор и память могут обработать данные).

```

root@Ubuntu:~# ./test.sh
none [mq-deadline] bfq kyber
[none] mq-deadline bfq kyber

/dev/sda:
Timing cached reads:   20648 MB in  1.99 seconds = 10379.92 MB/sec
Timing buffered disk reads: 266 MB in  3.00 seconds =  88.56 MB/sec
----
none [mq-deadline] bfq kyber

/dev/sda:
Timing cached reads:   22750 MB in  1.98 seconds = 11470.43 MB/sec
Timing buffered disk reads: 342 MB in  3.03 seconds = 112.83 MB/sec
----
none mq-deadline [bfq] kyber

/dev/sda:
Timing cached reads:   19358 MB in  1.99 seconds = 9744.65 MB/sec
Timing buffered disk reads: 466 MB in  3.04 seconds = 153.43 MB/sec
----
none mq-deadline bfq [kyber]

/dev/sda:
Timing cached reads:   18190 MB in  1.99 seconds = 9161.35 MB/sec
Timing buffered disk reads: 560 MB in  3.01 seconds = 186.12 MB/sec
----
root@Ubuntu:~# █

```

Рисунок 3 – Пример запуска скрипта

Таблица 1 – Результаты запуска скрипта (5 раз). Время чтения из кэша и с диска (Мб/с)

	none		mq-deadline		bfq		kyber	
Timing reads (MB/sec)	Cached	Buffered disk	Cached	Buffered disk	Cached	Buffered disk	Cached	Buffered disk
1	10379,92	88,56	11470,43	112,83	9744,65	153,43	9161,35	186,12
2	10454,39	179,98	8951,49	229,67	11847,62	236,45	13747,08	286,33
3	11412,27	263,00	9873,64	271,30	9270,26	252,26	12148,11	274,38
4	10406,75	251,13	12866,14	279,70	9419,95	284,83	13273,25	261,03
5	12947,04	259,45	12289,53	255,30	12029,76	269,31	13288,13	288,54
Средние значения, Мб/с	11120,07	208,42	11090,25	229,76	10462,45	239,26	12323,58	259,28

Проанализировав таблицу, можно заметить, что в среднем больше всего данных в секунду считывает `kyber`. Все остальные планировщики справляются чуть хуже, чем `kyber`, но, по сравнению друг с другом, они не сильно отличаются. Кто-то больше считывает за секунду из кэша, а кто-то – с диска.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены принципы работы актуальных планировщиков ввода-вывода, проведено тестирование их на чтение данных из кэша и с диска.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. <https://habr.com/ru/post/81504/>
2. <https://xakep.ru/2014/05/11/input-out-linux-planning/>
3. <https://habr.com/ru/company/selectel/blog/346844/>
4. <https://habrahabr.ru/post/337102/>
5. https://www.opennet.ru/base/sys/linux_shedulers.txt.html
6. <http://books.gigatux.nl/mirror/kerneldevelopment/0672327201/ch13lev1sec5.html>
7. <https://www.kernel.org/doc/Documentation/block/cfq-iosched.txt>
8. <https://www.kernel.org/doc/Documentation/block/bfq-iosched.txt>
9. <https://web.archive.org/web/20080606005055/http://www.dcs.ed.ac.uk/home/stg/pub/D/disk.html>
10. <https://www.kernel.org/doc/html/latest/block/bfq-iosched.html>
11. <https://www.geeksforgeeks.org/hdparm-command-in-linux-with-examples/>
12. https://wiki.archlinux.org/title/Hdparm_%28Русский%29